INSTITUTO TECNOLÓGICO DE LAS AMÉRICAS

Las Americas Institute of Technology

Integrante del grupo

Hector Jhoan Martinez Mendoza

Matricula

20231789

Carrera

Desarrollo de Software

Curso

Programación Paralela

Tema

Análisis de ventas usando descomposición de Datos

Fecha de entrega

4/25/2025

Índice

Índice **2**
1. Introducción 3
Objetivo general: 3
Objetivos específicos: 3
2. Descripción del Problema **3**
3. Cumplimiento de los Requisitos del Proyecto **4**
4. Diseño de la Solución **4**
Arquitectura general del sistema 4
Estrategia de paralelización utilizada 5
Manejo de datos y flujo de ejecución 5
Herramientas y tecnologías utilizadas 5
5. Implementación Técnica **6**
6. Evaluación de Desempeño **8**
7. Conclusión **9**
8. Anexos **10**
1. Requisitos del sistema 10
2. Descarga del proyecto 10
3. Manejo de los archivos CSV 10
4. Ejecución del sistema 11
5. Validación de resultados 11

1. Introducción

Este proyecto consiste en el desarrollo de un sistema para analizar grandes volúmenes de datos de ventas utilizando programación paralela en C#. La idea principal es comparar cómo se comporta el sistema cuando procesa la información de manera secuencial versus cuando aprovecha múltiples núcleos del procesador.

Se eligió este tema porque hoy en día casi todo sistema trabaja con grandes cantidades de datos, y el tiempo de respuesta se vuelve crítico. No es lo mismo procesar 100 mil registros que varios millones, y ahí es donde entra el paralelismo como una posible solución para mejorar el rendimiento.

Durante el desarrollo, nos dimos cuenta de que no todo tipo de cálculo se beneficia del paralelismo. Por eso, el proyecto evolucionó desde un análisis simple (sumas y promedios) hacia una simulación financiera más completa por cada registro, lo que permitió que el uso de múltiples núcleos tuviera un impacto real.

Objetivo general:

Evaluar el uso del paralelismo en el análisis de datos de ventas para mejorar el rendimiento del sistema.

Objetivos específicos:

- Procesar archivos CSV de gran tamaño
- Implementar una versión secuencial y otra paralela del análisis
- Comparar tiempos de ejecución
- Medir speedup y eficiencia
- Identificar en qué casos el paralelismo vale la pena y en cuáles no

2. Descripción del Problema

En muchos sistemas actuales, especialmente en áreas como comercio electrónico y análisis financiero, es común trabajar con grandes volúmenes de datos que deben ser procesados en el menor tiempo posible. Estos datos suelen incluir millones de registros que contienen información sobre transacciones, productos, clientes y valores monetarios.

El problema principal radica en que, a medida que el tamaño del dataset crece, el tiempo necesario para analizar esta información de forma secuencial aumenta considerablemente. Esto puede generar retrasos en la obtención de resultados, afectando la toma de decisiones en tiempo real o casi real.

En este contexto, surge la necesidad de utilizar técnicas que permitan mejorar el rendimiento del procesamiento de datos. Una de estas técnicas es el paralelismo, que consiste en dividir el trabajo en múltiples tareas que se ejecutan simultáneamente utilizando los diferentes núcleos del procesador.

El proyecto se enfoca en analizar un conjunto de datos de ventas y calcular métricas financieras relevantes, como ingresos totales, márgenes estimados, clasificación de ventas y simulaciones de escenarios. Estas operaciones requieren recorrer grandes cantidades de datos y realizar cálculos sobre cada registro, lo que representa un caso adecuado para evaluar el uso de procesamiento paralelo.

El reto consiste en determinar si el uso de múltiples núcleos realmente mejora el rendimiento del sistema, y bajo qué condiciones el paralelismo se convierte en una solución eficiente frente al enfoque secuencial tradicional.

3. Cumplimiento de los Requisitos del Proyecto

Ejecución simultánea de múltiples tareas:
El sistema divide el conjunto de datos en múltiples partes que pueden ser procesadas de manera independiente. Cada segmento del dataset es asignado a una tarea diferente, permitiendo que varios registros se analicen al mismo tiempo utilizando distintos núcleos del procesador. Esto asegura una ejecución concurrente real del análisis.

Necesidad de compartir datos entre tareas:
Aunque cada tarea trabaja sobre una porción distinta del dataset, todas contribuyen a un resultado global. Esto implica que los resultados parciales generados por cada hilo deben combinarse para obtener métricas finales como ingresos totales, márgenes, conteos y promedios. Por lo tanto, existe una necesidad clara de agregación de datos compartidos al finalizar el procesamiento.

Capacidad de escalar con más recursos:
La solución permite variar la cantidad de núcleos utilizados en la ejecución paralela, lo que hace posible observar cómo el sistema responde al aumento de recursos. A medida que se incrementa el número de procesadores, se pueden ejecutar más tareas simultáneamente, reduciendo el tiempo de procesamiento hasta cierto punto, lo que demuestra la escalabilidad del enfoque.

Inclusión de métricas de evaluación del rendimiento:
El sistema mide y compara el tiempo de ejecución tanto en modo secuencial como en paralelo. Además, calcula métricas clave como speedup y eficiencia, permitiendo analizar objetivamente el impacto del paralelismo y determinar si realmente mejora el rendimiento en cada escenario.

Aplicación a un problema del mundo real:
El análisis de datos de ventas es un caso práctico ampliamente utilizado en empresas que manejan grandes volúmenes de información. Procesar rápidamente estos datos permite obtener indicadores financieros, evaluar desempeño y tomar decisiones estratégicas. Este proyecto simula ese escenario, aplicando paralelismo para mejorar el tiempo de respuesta en el análisis.

4. Diseño de la Solución

Arquitectura general del sistema

El sistema se diseñó de forma simple pero organizada, separando las responsabilidades principales para que el código fuera más fácil de entender y de modificar.

Básicamente, el programa se divide en varias partes:

- Una encargada de manejar los archivos, que se ocupa de leer el CSV y mostrar la información básica del dataset.
- Otra parte que controla la ejecución del análisis, tanto en modo secuencial como en paralelo.
- Un módulo donde están los cálculos que se le aplican a cada registro (ingresos, costos, margen, score, etc.).
- Y finalmente una parte que calcula las métricas de rendimiento para comparar los resultados.

La idea detrás de esta separación fue evitar tener todo el código mezclado en un solo lugar. Así, si se quiere cambiar cómo se calculan los datos o cómo se mide el rendimiento, se puede hacer sin romper todo lo demás.

Estrategia de paralelización utilizada

Para resolver el problema se utilizó la estrategia de **descomposición de datos**.

Esto significa que, en lugar de procesar todo el dataset de forma lineal, se divide en varias partes y cada una se procesa de manera independiente. Como cada registro de venta no depende de otro, se pueden analizar en paralelo sin problemas.

En este proyecto, esa idea se aplicó de la siguiente forma:

- Se carga todo el dataset en memoria.
- Luego se divide la lista de registros en segmentos.
- Cada segmento es procesado por una tarea distinta.
- Todas las tareas se ejecutan al mismo tiempo usando varios núcleos del procesador.
- Al final, se combinan los resultados parciales para obtener el resultado total.

Este enfoque funciona bien porque el análisis se hace por registro, y no hay dependencia entre ellos. Eso evita problemas de sincronización complejos y hace que el paralelismo sea más eficiente.

Manejo de datos y flujo de ejecución

El flujo del programa es bastante directo:

Primero, el usuario selecciona el archivo que quiere analizar. Luego el sistema muestra algunos datos básicos del archivo, como tamaño y cantidad de registros, para confirmar que se está trabajando con el dataset correcto.

Después de eso, el archivo se carga completamente en memoria. Esto fue una decisión importante, porque permite que el tiempo de lectura del archivo no afecte las mediciones del rendimiento del análisis.

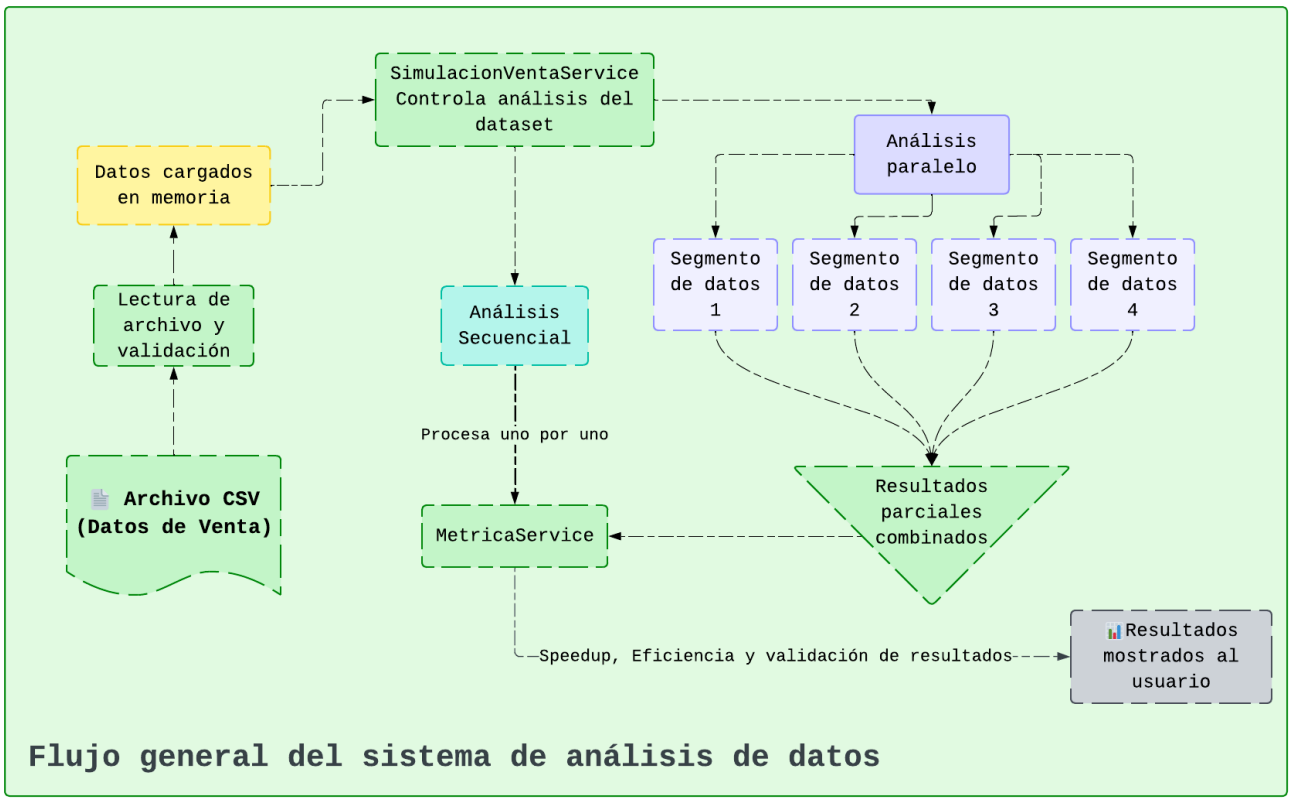
Una vez los datos están en memoria, se ejecuta primero el análisis secuencial, y luego el análisis paralelo utilizando diferentes cantidades de núcleos. Finalmente, se muestran los resultados junto con las métricas de rendimiento.

Herramientas y tecnologías utilizadas

Para el desarrollo del proyecto se utilizaron herramientas bastante estándar, pero elegidas con intención:

- **C# y .NET:**
  Se utilizaron porque ofrecen buen soporte para programación paralela y permiten trabajar con estructuras de datos de forma eficiente. Además, ya era el lenguaje base del curso.

- **Task Parallel Library (TPL):**
  Se utilizó porque facilita la creación y manejo de tareas paralelas sin tener que trabajar directamente con hilos de bajo nivel. Esto hace que el código sea más limpio y más fácil de controlar.
- **CSVHelper:** Se utilizó para leer los archivos CSV de forma más sencilla y segura. Leer archivos grandes manualmente puede volverse complicado, y esta librería ayuda a manejar conversiones de tipos y estructura de datos sin tanto problema.
- **Stopwatch:**
  Se utilizó para medir el tiempo de ejecución de manera precisa. Era importante poder comparar claramente la ejecución secuencial con la paralela.



Flujo general del sistema de análisis de datos

5. Implementación Técnica

El proyecto fue desarrollado en C# utilizando .NET como plataforma principal. La aplicación funciona desde consola y permite seleccionar un archivo CSV de ventas para analizarlo de forma secuencial y paralela.

La estructura del código se organizó en varios componentes para mantener una separación clara de responsabilidades. El servicio de archivos se encarga de listar los datasets disponibles, cargar el CSV y obtener información básica del archivo. El servicio de simulación controla la ejecución del análisis secuencial y paralelo. La calculadora de ventas contiene los cálculos aplicados a cada registro, y el servicio de métricas se encarga de calcular speedup, eficiencia y validar que los resultados sean equivalentes.

El análisis se realiza sobre registros de ventas cargados en memoria. Cada registro contiene datos como precio, cantidad, costo de envío y valor pagado. Con esa información, el sistema calcula métricas como ingreso total, costo estimado, margen, clasificación de la venta y un score comercial. Estos cálculos permiten que el procesamiento tenga una carga computacional suficiente para evaluar correctamente el beneficio del paralelismo.

En la versión secuencial, el sistema recorre todos los registros uno por uno utilizando un solo flujo de ejecución. Este resultado sirve como referencia para comparar el comportamiento de la versión paralela.

En la versión paralela, el conjunto de registros se divide en segmentos. Cada segmento es procesado de manera independiente, y al final los resultados parciales se combinan para obtener un único resultado global. Esta estrategia evita que varios procesos modifiquen el mismo resultado al mismo tiempo, reduciendo el riesgo de condiciones de carrera.

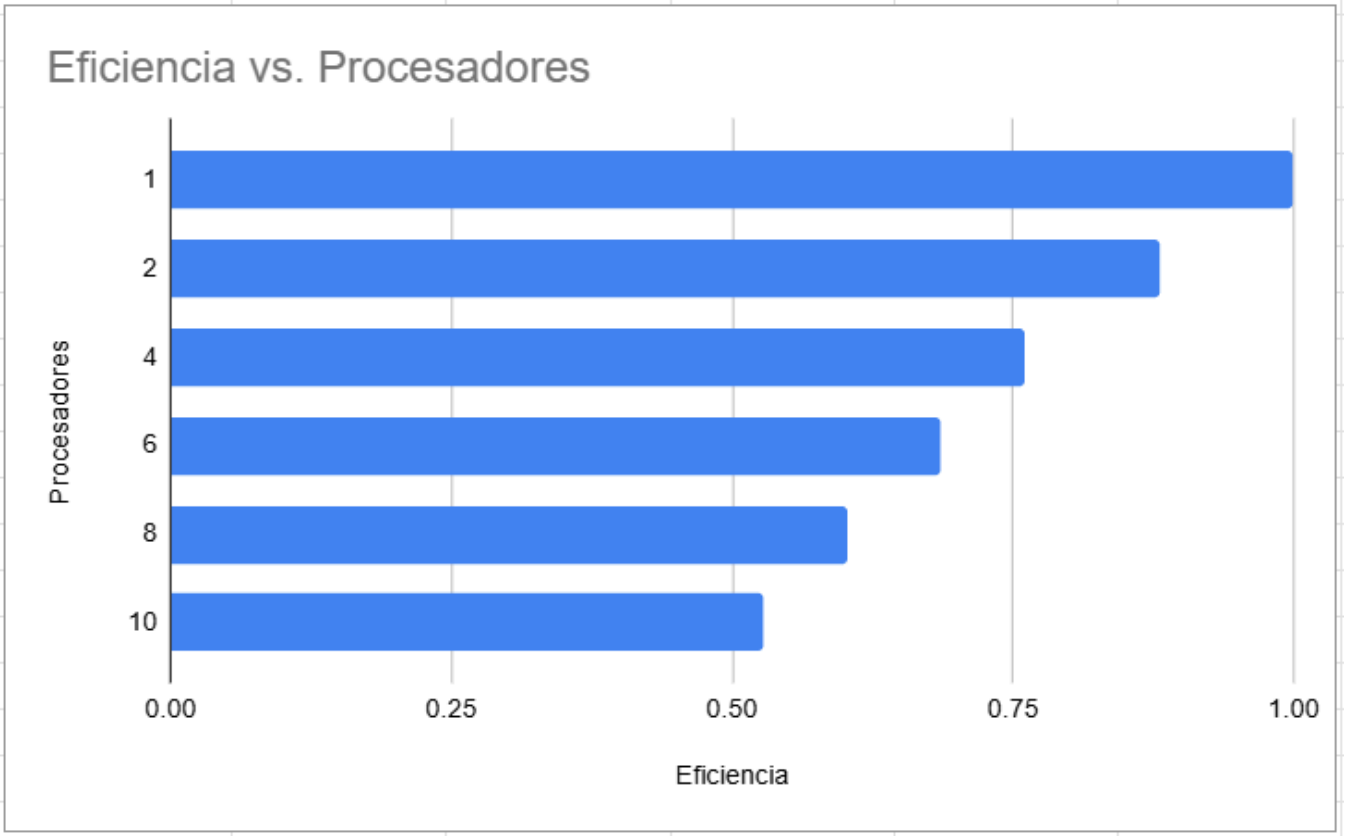
Para la medición del rendimiento se utilizó Stopwatch, registrando el tiempo de ejecución de cada enfoque. A partir de esos tiempos se calcula el speedup, que indica cuántas veces más rápida fue la ejecución paralela, y la eficiencia, que indica qué tan bien se aprovecharon los procesadores utilizados.

Una decisión técnica importante fue separar el tiempo de carga del archivo CSV del tiempo de procesamiento. Inicialmente, el tiempo de lectura del archivo afectaba demasiado las mediciones, ya que cargar un CSV grande depende más del disco que del procesador. Por esta razón, el archivo se carga primero en memoria, y luego se ejecutan las pruebas secuencial y paralela sobre los mismos datos.

También se implementó una validación entre los resultados secuenciales y paralelos. Como algunos cálculos utilizan valores decimales de tipo double, se utiliza una comparación con tolerancia relativa en lugar de igualdad exacta. Esto evita marcar como incorrectos resultados que son equivalentes, pero que pueden variar mínimamente por el orden de las operaciones en paralelo.

En conjunto, la implementación permite comparar de forma clara ambos enfoques y demostrar en qué condiciones el paralelismo realmente mejora el rendimiento del análisis de datos.

6. Evaluación de Desempeño



Modo de ejecución	Tiempo (ms)	Speedup	Eficiencia
Secuencial	12,072.00	1.00x	1.00
Paralelo 2 procesadores	6,842.00	1.76x	0.88
Paralelo 4 procesadores	3,953.00	3.05x	0.76
Paralelo 6 procesadores	2938	4.11x	0.6848
Paralelo 8 procesadores	2508	4.81x	0.6017
Paralelo 10 procesadores	2292	5.27x	0.5267

Tabla de comparación entre los tiempos secuenciales, paralelos, speedup y eficiencia entre cada iteración.

Métrica	Valor
Total revenue	1,917,332,455.59
Costo estimado total	492,968,272.79
Margen total	1,424,364,182.80
Promedio revenue	383.47
Score promedio	3168.34

Los resultados secuenciales y Paralelos fueron los mismos

Para comparar el rendimiento, se ejecutó el mismo análisis sobre los datasets utilizando dos enfoques: secuencial y paralelo. En ambos casos se trabajó con exactamente los mismos datos cargados en memoria, asegurando que la única diferencia fuera la forma en que se procesaban. Se midió el tiempo de ejecución de cada corrida y, a partir de ahí, se calcularon métricas como speedup y eficiencia para evaluar cuánto mejoraba el rendimiento al utilizar múltiples procesadores.

Los resultados mostraron que el paralelismo sí mejora el tiempo de ejecución de forma clara, especialmente cuando el dataset es grande y los cálculos son más complejos. Sin embargo, también se observó que esta mejora no crece de forma proporcional con la cantidad de procesadores. Existe un punto donde agregar más núcleos sigue reduciendo el tiempo, pero la eficiencia baja, lo que indica que no siempre vale la pena utilizar todos los recursos disponibles. En este proyecto, el mejor balance entre rendimiento y eficiencia se encontró generalmente entre 4 y 6 procesadores.

Uno de los principales cuellos de botella identificados fue la carga del archivo CSV. Al inicio, el tiempo de lectura del archivo representaba una parte significativa del tiempo total de ejecución, lo que afectaba la comparación entre el modo secuencial y el paralelo. Esto ocurría porque la lectura de archivos depende del disco y no del procesador, por lo que no se beneficia del paralelismo. Para solucionar esto, se decidió separar completamente la carga del dataset del proceso de análisis, cargando todos los datos en memoria antes de iniciar las pruebas.

Otra limitación importante fue que los cálculos simples no generaban un beneficio real al ejecutarse en paralelo. Operaciones como sumas o conteos tienen un costo computacional bajo, por lo que el overhead de dividir el trabajo y coordinar múltiples tareas terminaba reduciendo la eficiencia. Por esta razón, se modificó el enfoque del proyecto incorporando cálculos más complejos por cada registro, como estimaciones de costos, márgenes y un score financiero. Esto permitió aumentar la carga de trabajo del sistema y lograr que el paralelismo tuviera un impacto real en el rendimiento.

7. Conclusión

El desarrollo de este proyecto permitió demostrar de forma clara que el paralelismo puede mejorar significativamente el tiempo de ejecución en el procesamiento de grandes volúmenes de datos, siempre que el problema esté bien diseñado. Al comparar la ejecución secuencial con la paralela, se evidenció que dividir el trabajo en múltiples partes y procesarlas simultáneamente reduce el tiempo total, especialmente cuando los cálculos por registro tienen un costo computacional relevante.

No obstante, también se comprobó que el paralelismo no siempre escala de manera ideal. A medida que aumenta la cantidad de procesadores, la eficiencia tiende a disminuir debido a la sobrecarga de coordinación entre tareas. Esto demuestra que existe un punto óptimo donde se obtiene el mejor balance

entre rendimiento y uso de recursos, y que utilizar todos los núcleos disponibles no siempre es la mejor decisión.

Finalmente, el proyecto permitió entender que el beneficio del paralelismo depende directamente de la naturaleza del problema. Separar la carga de datos del procesamiento y aumentar la complejidad de los cálculos fueron decisiones clave para obtener resultados más representativos. En conjunto, se concluye que el paralelismo es una solución efectiva, pero debe aplicarse de forma estratégica, considerando tanto sus ventajas como sus limitaciones en escenarios reales.

8. Anexos

1. Requisitos del sistema

Para ejecutar el proyecto es necesario tener instalado .NET (versión 6.0 o superior) en el equipo. Se recomienda utilizar Visual Studio o Visual Studio Code para facilitar la ejecución, aunque también puede ejecutarse directamente desde la consola del sistema.

2. Descarga del proyecto

El proyecto se encuentra disponible en el siguiente repositorio:

https://github.com/HectorMMartinez/Proyecto-Final-Desc

Puede descargarse en formato ZIP o clonarse utilizando Git. Una vez descargado, se debe extraer el contenido en una carpeta local para poder trabajar con él.

3. Manejo de los archivos CSV

Debido al tamaño de los datasets utilizados en el proyecto, los archivos CSV no se almacenan directamente sin compresión dentro del repositorio. En su lugar, se incluyen en formato comprimido para facilitar su almacenamiento y descarga.

Para mantener el repositorio ligero, solo se incluyen los datasets más pequeños como base. Antes de ejecutar el sistema, es necesario descomprimir estos archivos y colocarlos en la carpeta:

data/input/

Una vez descomprimidos, el sistema los reconocerá automáticamente como opciones disponibles para su análisis.

En caso de que se deseen utilizar datasets más grandes, estos pueden generarse fácilmente a partir de los archivos base. Para hacerlo, basta con abrir una consola dentro de la carpeta `data/input/` y ejecutar un comando que replique el contenido del archivo varias veces.

Por ejemplo, utilizando PowerShell:

```
1..5 | ForEach-Object { Get-Content Ventas_100k.csv } | Set-Content Ventas_500k.csv
```

Este comando genera un archivo más grande repitiendo el contenido del dataset original. El número inicial (en este caso 5) puede modificarse para crear archivos de mayor o menor tamaño según se necesite.

De esta forma, es posible escalar el volumen de datos para realizar pruebas más exigentes sin necesidad de incluir archivos pesados directamente en el repositorio.

4. Ejecución del sistema

El sistema puede ejecutarse abriendo el proyecto en Visual Studio y presionando ejecutar, o desde la consola ubicándose en la carpeta del proyecto y ejecutando el comando correspondiente de .NET.

Al iniciar, el programa solicita al usuario seleccionar un archivo de datos y elegir el modo de ejecución. A partir de ahí, el sistema realiza el análisis secuencial y paralelo, mostrando en pantalla los resultados y las métricas de rendimiento.

5. Validación de resultados

El sistema incluye una validación automática que compara los resultados del análisis secuencial con los del análisis paralelo. Debido al uso de valores decimales, se aplica una pequeña tolerancia para evitar diferencias mínimas causadas por el orden de las operaciones.

Cuando los resultados coinciden correctamente, el sistema indica que la validación es satisfactoria, asegurando que el paralelismo no afecta la precisión de los cálculos.

```
Seleccione el archivo a procesar:
1. Ventas_153mb.csv
2. Ventas_15mb.csv
3. Ventas_3gb.csv
4. Ventas_700mb.csv
5. Ventas_76mb.csv

Ingrese una opción: 4

Archivo seleccionado:
C:\Users\hevto\source\repos\ProyectoFinalParalelo\Proyecto-Final-Desc\data\input\Ventas_700mb.csv

=== METADATA DEL ARCHIVO ===
Nombre: Ventas_700mb.csv
Ruta: C:\Users\hevto\source\repos\ProyectoFinalParalelo\Proyecto-Final-Desc\data\input\Ventas_700mb.csv
Tamaño: 750.28 MB
Cantidad de registros: 5000000
Cantidad de columnas: 12

Procesadores lógicos disponibles: 16

Modo:
1. Automático
2. Personalizado
Opción: 1

Cargando CSV en memoria...
Registros cargados: 5000000
Tiempo de carga: 25033 ms

=== SIMULACION SECUENCIAL ===

=== RESULTADO SECUENCIAL ===
Total registros: 5000000
Total revenue: 1917332455.58
Costo estimado total: 492968272.79
Margen estimado total: 1424364182.83
Promedio revenue: 383.47
Promedio margen: 284.87
Ventas rentables: 4927050
Ventas en riesgo: 72950
Ventas con pérdida: 0
Score total: 15841718001.60
Score promedio: 3168.34
Score máximo: 63213.90
Mejor escenario de descuento: 0%
Margen del mejor escenario: 19139.43
Tiempo: 13914 ms
```

```
=== SIMULACION PARALELA (2 procesadores) ===

=== RESULTADO PARALELO (2) ===
Total registros: 5000000
Total revenue: 1917332455.59
Costo estimado total: 492968272.79
Margen estimado total: 1424364182.80
Promedio revenue: 383.47
Promedio margen: 284.87
Ventas rentables: 4927050
Ventas en riesgo: 72950
Ventas con pérdida: 0
Score total: 15841718001.58
Score promedio: 3168.34
Score máximo: 63213.90
Mejor escenario de descuento: 0%
Margen del mejor escenario: 19139.43
Tiempo: 8062 ms
Speedup: 1.73x
Eficiencia: 86.29%
Validación: Correcta
Conclusión: Vale la pena: eficiencia buena.

=== SIMULACION PARALELA (4 procesadores) ===

=== RESULTADO PARALELO (4) ===
Total registros: 5000000
Total revenue: 1917332455.59
Costo estimado total: 492968272.79
Margen estimado total: 1424364182.80
Promedio revenue: 383.47
Promedio margen: 284.87
Ventas rentables: 4927050
Ventas en riesgo: 72950
Ventas con pérdida: 0
Score total: 15841718001.58
Score promedio: 3168.34
Score máximo: 63213.90
Mejor escenario de descuento: 0%
Margen del mejor escenario: 19139.43
Tiempo: 4569 ms
Speedup: 3.05x
Eficiencia: 76.13%
Validación: Correcta
Conclusión: Vale la pena: eficiencia buena.

=== SIMULACION PARALELA (6 procesadores) ===

=== RESULTADO PARALELO (6) ===
Total registros: 5000000
Total revenue: 1917332455.59
Costo estimado total: 492968272.79
Margen estimado total: 1424364182.80
Promedio revenue: 383.47
Promedio margen: 284.87
Ventas rentables: 4927050
Ventas en riesgo: 72950
Ventas con pérdida: 0
Score total: 15841718001.58
Score promedio: 3168.34
Score máximo: 63213.90
Mejor escenario de descuento: 0%
Margen del mejor escenario: 19139.43
Tiempo: 3481 ms
Speedup: 4.00x
Eficiencia: 66.62%
Validación: Correcta
Conclusión: Aceptable: todavía puede considerarse.
```

```
=== SIMULACION PARALELA (8 procesadores) ===

=== RESULTADO PARALELO (8) ===
Total registros: 5000000
Total revenue: 1917332455.59
Costo estimado total: 492968272.79
Margen estimado total: 1424364182.80
Promedio revenue: 383.47
Promedio margen: 284.87
Ventas rentables: 4927050
Ventas en riesgo: 72950
Ventas con pérdida: 0
Score total: 15841718001.58
Score promedio: 3168.34
Score máximo: 63213.90
Mejor escenario de descuento: 0%
Margen del mejor escenario: 19139.43
Tiempo: 3012 ms
Speedup: 4.62x
Eficiencia: 57.74%
Validación: Correcta
Conclusión: No vale la pena: eficiencia baja.

=== SIMULACION PARALELA (10 procesadores) ===

=== RESULTADO PARALELO (10) ===
Total registros: 5000000
Total revenue: 1917332455.59
Costo estimado total: 492968272.79
Margen estimado total: 1424364182.80
Promedio revenue: 383.47
Promedio margen: 284.87
Ventas rentables: 4927050
Ventas en riesgo: 72950
Ventas con pérdida: 0
Score total: 15841718001.58
Score promedio: 3168.34
Score máximo: 63213.90
Mejor escenario de descuento: 0%
Margen del mejor escenario: 19139.43
Tiempo: 2487 ms
Speedup: 5.59x
Eficiencia: 55.95%
Validación: Correcta
Conclusión: No vale la pena: eficiencia baja.

=== SIMULACION PARALELA (12 procesadores) ===

=== RESULTADO PARALELO (12) ===
Total registros: 5000000
Total revenue: 1917332455.59
Costo estimado total: 492968272.79
Margen estimado total: 1424364182.80
Promedio revenue: 383.47
Promedio margen: 284.87
Ventas rentables: 4927050
Ventas en riesgo: 72950
Ventas con pérdida: 0
Score total: 15841718001.58
Score promedio: 3168.34
Score máximo: 63213.90
Mejor escenario de descuento: 0%
Margen del mejor escenario: 19139.43
Tiempo: 2453 ms
Speedup: 5.67x
Eficiencia: 47.27%
Validación: Correcta
Conclusión: No vale la pena: eficiencia baja.
```

```
=== SIMULACION PARALELA (16 procesadores) ===

=== RESULTADO PARALELO (16) ===
Total registros: 5000000
Total revenue: 1917332455.59
Costo estimado total: 492968272.79
Margen estimado total: 1424364182.80
Promedio revenue: 383.47
Promedio margen: 284.87
Ventas rentables: 4927050
Ventas en riesgo: 72950
Ventas con pérdida: 0
Score total: 15841718001.58
Score promedio: 3168.34
Score máximo: 63213.90
Mejor escenario de descuento: 0%
Margen del mejor escenario: 19139.43
Tiempo: 2213 ms
Speedup: 6.29x
Eficiencia: 39.30%
Validación: Correcta
Conclusión: No vale la pena: eficiencia baja.

¿Desea analizar otro archivo? (s/n): |
```